

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 05 · LECTURE NOTES

Dot Products and Projection

The operation every shading calculation is made of.

MODULE OUTCOMES

- Compute dot products and interpret them geometrically.
- Derive the projection formula.
- Explain orthogonality and orthonormal bases.
- Apply Gram-Schmidt.
- Recognise dot products in shading code.

1 Two definitions

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + a_3 b_3 = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

- **The algebraic definition is componentwise multiply and sum** — cheap to compute, and **it tells you nothing by itself** about what the number means.
- **The geometric meaning is $|\mathbf{a}||\mathbf{b}|\cos\theta$, and the equality of these two expressions is essentially the law of cosines** — **which is worth proving once**, because it is the bridge between the arrow picture and the coordinate arithmetic.
- **So for unit vectors, $\mathbf{a} \cdot \mathbf{b}$ is the cosine of the angle between them** — **which is exactly why normalised vectors are everywhere in shading code**: the dot product becomes the angle term directly, with no trigonometry and no division.
- **The agreement of the two definitions is the content of this section; either one alone is just a formula**, and the whole usefulness of the operation comes from a cheap computation having a geometric meaning.

AND THE SIGN ALONE ANSWERS THE QUESTIONS YOU ASK MOST OFTEN

Positive means the angle is acute; zero means exactly perpendicular; negative means obtuse — and you frequently need nothing more than that, which makes the operation cheaper still.

Is the light in front of this surface? $n \cdot l > 0$. Is this face pointing toward the camera? $n \cdot v > 0$ — which is backface culling, and it is one dot product and one comparison per triangle.

Which is why the dot product appears in every shading model — Lambertian diffuse is literally $\max(0, n \cdot l)$ times the light colour, and that is the entire formula, which surprises people who expected something more elaborate.

And the clamp matters: a negative dot product means the light is behind the surface, and **without the max you get negative light** — which subtracts from other contributions and produces black rims. **This is the first bug everybody writes**, and it is worth writing once deliberately.

2 Projection

QUESTION: how much of a points along b?

THE ANSWER has to be a multiple of b, say $c \cdot b$, and the leftover $(a - c \cdot b)$ has to be perpendicular to b — that is what "the component along b" means.

SO IMPOSE IT:

$$\begin{aligned}(a - c \cdot b) \cdot b &= 0 \\ a \cdot b - c (b \cdot b) &= 0 \\ c &= (a \cdot b) / (b \cdot b)\end{aligned}$$

THEREFORE

$$\text{proj}_b(a) = [(a \cdot b) / (b \cdot b)] \cdot b$$

AND IF b IS A UNIT VECTOR, $b \cdot b = 1$, so

$$\text{proj}_b(a) = (a \cdot b) \cdot b$$

which is why everything gets normalised first.

THE DERIVATION IS THREE LINES and it is worth redoing rather than recalling: the condition "the leftover is perpendicular" IS the definition, and everything else is algebra.

'The leftover is perpendicular' is the definition, and the formula follows in three lines — which is why it never needs memorising, and why getting it slightly wrong from memory is avoidable. **This decomposition of a vector into a part along b and a part perpendicular to it is used constantly in CSCE 649**: a collision response splits velocity into normal and tangential components and treats them differently, which is this formula twice.

3 Orthogonality

- **Coordinates become plain dot products** — the i -th coordinate of $\mathbf{v}$ is $\mathbf{v} \cdot \mathbf{e}_i$, with no linear system to solve, which in a general basis would require a solve or a stored inverse.
- **The inverse of the basis matrix is its transpose** (Module 04 §2) — free to compute, and numerically exact rather than approximately correct.
- **Lengths and angles are preserved by the transform** — which is precisely what makes a rotation a rotation rather than a general invertible map, and is the property Module 09 §3 detects with a determinant.
- **And decomposition becomes trivial: any vector is the sum of its projections onto the basis vectors, which is §2 applied three times** and no more difficult than that.
- **So the question is always: how do I get an orthonormal basis in the first place?** — and Gram-Schmidt is the answer (§4). Finding coordinates in an orthonormal basis is three dot products instead of a linear solve, which is the practical reason the condition is worth arranging deliberately.

4 Gram-Schmidt

TAKE EACH VECTOR, SUBTRACT ITS PROJECTIONS ONTO THE ONES ALREADY DONE, AND NORMALISE

That is the entire algorithm — process the vectors in order, each time removing whatever part lies in the span of the ones already processed — **and each removal step is §2's projection, so there is nothing new to learn.**

Which gives you an orthonormal basis spanning exactly the same space as the one you started with — so you can always arrange to be in §3's easy case, at the cost of a few dot products.

And the graphics use is constant: building a tangent frame around a surface normal for importance sampling (CSCE 647 Module 07, where you need an orthonormal basis with the normal as one axis and do not care which two vectors fill out the rest), or **re-orthonormalising a camera or bone frame that has drifted from accumulated error** (Module 04 §2's numerical point).

Plus the numerical caveat: the naive formulation loses orthogonality badly on nearly-dependent input, because the subtractions are all computed against the original vectors — **the modified version subtracts progressively, updating as it goes, and is what you should actually implement.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
3Blue1Brown, chapter 9 (free)	§1 — the dot product's two definitions and why they agree, made visual.

Resource	What it covers for this module
MIT 18.06, the orthogonality lectures (free video)	§§2 to 4 — projection, orthonormal bases, and Gram-Schmidt, with the least-squares connection previewed (Module 12).
Immersive Math, chapter 4 (free)	§§1 and 2 — drag the vectors and watch the projection follow.
Pharr, Jakob & Humphreys — PBRT, the sampling chapters (free online)	§4's graphics use — orthonormal frames around a normal, which is CSCE 647's version of this material.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compute ten dot products** and predict each sign first.
- 2 **Prove the two definitions agree**, via the law of cosines.
- 3 **Implement backface culling** with a single dot product.
- 4 **Implement Lambertian shading**, and then remove the clamp and look at the result.
- 5 **Derive the projection formula** from the perpendicularity condition.
- 6 **Decompose a vector** into parallel and perpendicular parts relative to another.
- 7 **Find coordinates in an orthonormal basis** by dot products, and verify against a solve.
- 8 **Implement Gram-Schmidt**, naive and modified.
- 9 **Feed both nearly-dependent input** and measure the orthogonality loss.
- 10 **Build a tangent frame** around an arbitrary normal.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give both definitions of the dot product and say why they agree.
- 2 What is the dot product of two unit vectors?
- 3 What does the sign tell you, and name two uses.
- 4 Write the Lambertian formula and explain the clamp.
- 5 Derive the projection formula from perpendicularity.
- 6 What does it simplify to for a unit vector?
- 7 Name four things an orthonormal basis buys you.
- 8 How do you find coordinates in one?

- 9 Describe Gram-Schmidt in one sentence.
- 10 What is the numerical caveat, and what should you implement?